

Docker

Часть 1

Евгений Мисяков
Teamlead Devops Умскул



Проверка связи



Если у вас нет звука:

- убедитесь, что на вашем устройстве и на колонках включён звук
- обновите страницу вебинара (или закройте страницу и заново присоединитесь к вебинару)
- откройте вебинар в другом браузере
- перезагрузите компьютер (ноутбук) и заново попытайтесь зайти



Поставьте в чат:

- + если меня видно и слышно
- если нет

Евгений Мисяков

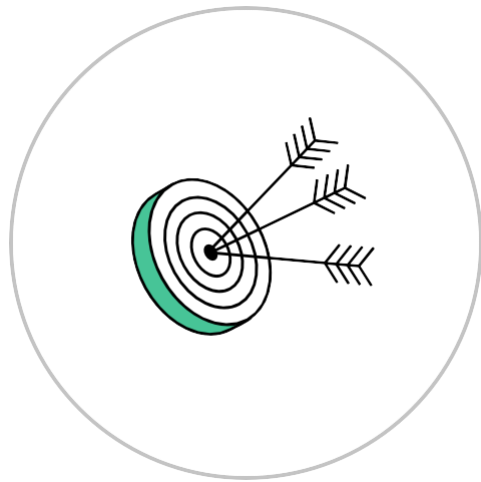
О спикере:

- SRE инженер в Нетологии



Цели занятия

- Узнать, что такое Docker и какие функции он выполняет
- Разобраться, что такое DockerHub, Dockerfile, Docker Image и Container
- Научиться устанавливать Docker на Debian
- Запустить тестовый контейнер hello-world
- Научиться управлять образами и контейнерами Docker



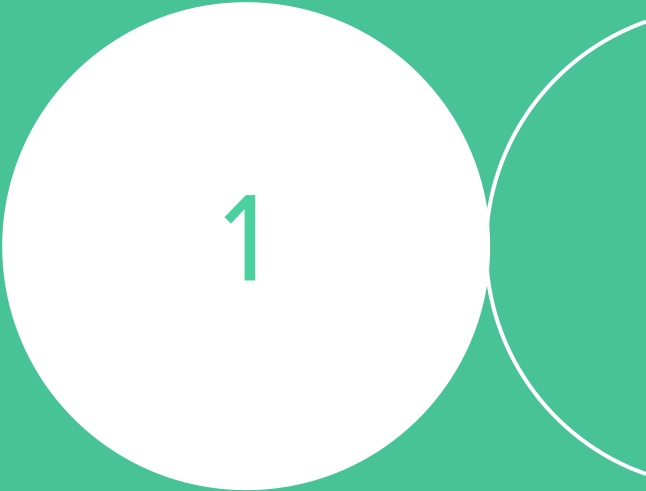
План занятия

- 1 Основные понятия
- 2 Компоненты экосистемы Docker
- 3 Dockerfile
- 4 Начало работы с Docker
- 5 Итоги
- 6 Домашнее задание



*Нажмите на нужный раздел для перехода

Основные понятия



1

Вспоминаем прошрое занятие

Вопрос: Что такое виртуализация?



Вспоминаем прошрое занятие

Вопрос: Что такое виртуализация?

Ответ: Виртуализация — это создание
изолированных окружений в рамках одного
физического устройства





Контейнер — это способ упаковать приложение и все его зависимости в единый образ. Этот образ запускается в изолированной среде, не влияющей на основную операционную систему

Отличие контейнеризации от виртуализации

Обе технологии являются абстракциями физического оборудования.

Но инструменты контейнеризации используют ядро хостовой ОС, по этой причине не обеспечивая такую высокую степень изоляции между контейнерами, как между виртуальными машинами в виртуализации.

Зачастую эти две технологии дополняют друг друга: в виртуальной среде вполне можно (и такое практикуется) развернуть контейнеры.



Docker — это платформа для разработки, доставки и запуска контейнеров с приложениями.

Docker

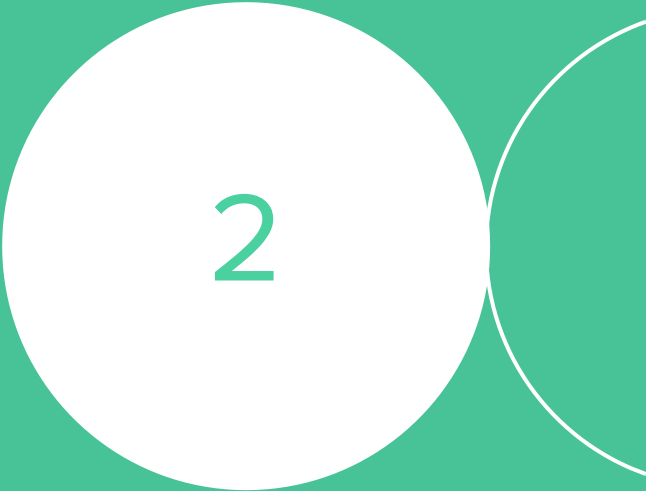
Docker позволяет:

- создавать контейнеры
- автоматизировать их запуск и развертывание
- управлять жизненным циклом

Основное назначение Docker

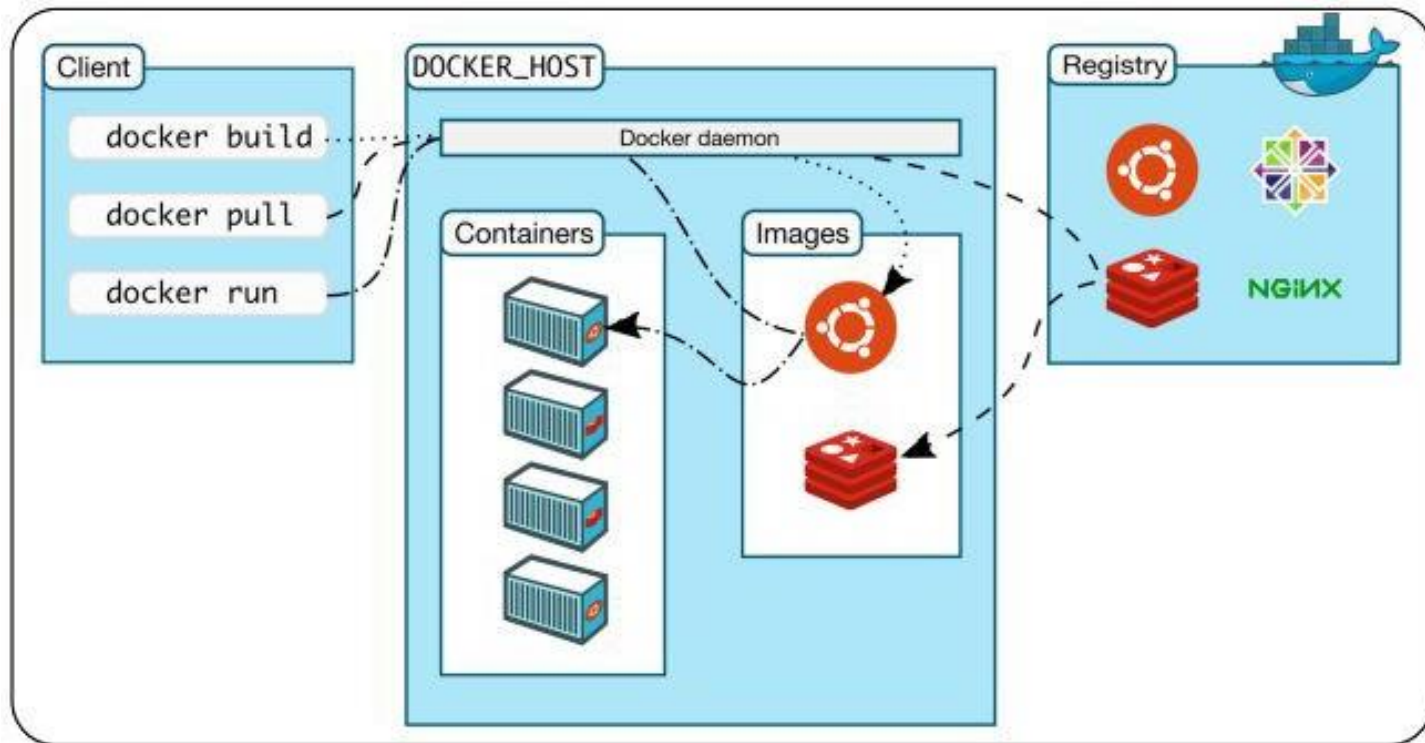
- Решает проблему с окружением, необходимым для работы приложения при разработке
- Исключает необходимость воссоздания окружения при переносе приложения с одного сервера на другой
- Запускает специальные образы, из которых вырезано почти всё, что не требуется для работы приложения (принцип наименьших привилегий в ИБ)

Компоненты экосистемы Docker



2

Архитектура Docker



Компоненты экосистемы Docker

Docker Daemon (Docker демон) — сервер контейнеров, входящий в состав программных средств Docker:

- управляет Docker-объектами
- может связываться с другими демонами

Docker Client/CLI (Docker клиент) — интерфейс взаимодействия пользователя с Docker-демоном.

- Клиент и Демон — важнейшие компоненты «движка» Докера (Docker Engine)
- Клиент Docker может взаимодействовать с несколькими демонами

Компоненты экосистемы Docker

- Docker Image (Docker образ) — шаблон, включающий зависимости, сведения, конфигурацию для дальнейшего развертывания и инициализации контейнера
- Dockerfile (Docker файл) — описание правил (манифест) для сборки образа
- Docker Container (Docker контейнер) — это легкий, автономный исполняемый пакет программного обеспечения, который включает в себя все необходимое для запуска приложения: код, среду выполнения, системные инструменты, системные библиотеки и настройки

Компоненты экосистемы Docker

Docker Image можно сравнить с аудиокассетой или диском. Вы покупаете подобный носитель и вне зависимости от проигрывателя, в который вы его вставляете, услышите одну и ту же песню.

По аналогии с этим, из тысяч Docker-серверов, запустивших один и тот же Docker Image, все дадут одинаковый результат.

В таком случае Docker Container можно сравнить с вставленной в магнитофон и играющей кассетой.

10 раз запустив один Docker Image, мы создадим 10 одинаковых Docker контейнеров.

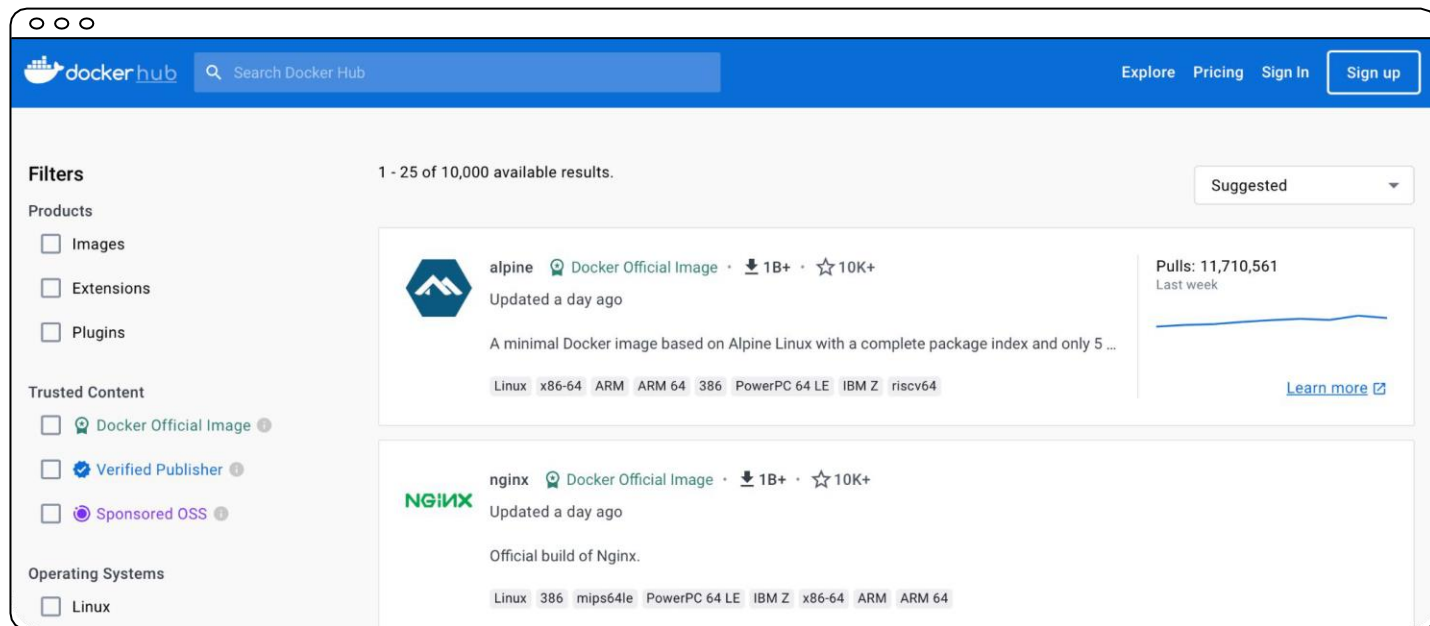
Компоненты экосистемы Docker

- Volume (Том хранения данных) — эмуляция файловой системы для осуществления операций чтения и записи. Она создается автоматически с контейнером, поскольку некоторые приложения осуществляют персистентное хранение данных
- Docker Registry (Реестр Docker контейнеров) — зарезервированный сервер, используемый для хранения docker-образов
- Docker Hub (Docker Хаб) — общедоступный и бесплатный репозиторий, предназначенный для хранения образов с различным программным обеспечением
- Docker Trusted Registry (Доверенный реестр Docker или DTR) — служба docker-реестра для инсталляции на локальном компьютере или сети компании. Есть аналоги: Nexus, Google Container Registry и прочие

Docker Hub

Docker Hub — публичное облачное хранилище от официальных разработчиков, куда можно загрузить свои Docker Image для последующей работы с ними.

Репозиторий в Docker Hub может быть как публичным, так и приватным



The screenshot displays the Docker Hub interface. At the top, there's a blue header with the Docker Hub logo, a search bar, and navigation links: 'Explore', 'Pricing', 'Sign In', and a 'Sign up' button. Below the header, the main content area shows search results. On the left, there's a 'Filters' sidebar with sections for 'Products' (Images, Extensions, Plugins), 'Trusted Content' (Official Image, Verified Publisher, Sponsored OSS), and 'Operating Systems' (Linux). The main results area shows '1 - 25 of 10,000 available results.' and a 'Suggested' dropdown. Two image cards are visible: 'alpine' (Docker Official Image) and 'nginx' (Docker Official Image). Each card includes the image icon, name, 'Docker Official Image' badge, download count (1B+), star count (10K+), update time ('Updated a day ago'), description, supported architectures (Linux, x86-64, ARM, ARM 64, 386, PowerPC 64 LE, IBM Z, riscv64), pull count (11,710,561), and a 'Learn more' link.

Filters

1 - 25 of 10,000 available results.

Suggested

Products

- ☐ Images
- ☐ Extensions
- ☐ Plugins

Trusted Content

- ☐ Docker Official Image
- ☐ Verified Publisher
- ☐ Sponsored OSS

Operating Systems

- ☐ Linux

alpine Docker Official Image · 1B+ · 10K+
Updated a day ago
A minimal Docker image based on Alpine Linux with a complete package index and only 5 ...
Linux x86-64 ARM ARM 64 386 PowerPC 64 LE IBM Z riscv64
Pulls: 11,710,561
Last week
[Learn more](#)

nginx Docker Official Image · 1B+ · 10K+
Updated a day ago
Official build of Nginx.
Linux 386 mips64le PowerPC 64 LE IBM Z x86-64 ARM ARM 64

Компоненты экосистемы Docker

- Docker Host (Docker хост) — среда, на которой запускается Docker Engine в виде системного демона для запуска контейнеров с программным обеспечением, указанным в Docker файле
- Docker Networks (Docker сети) — применяются для организации сетевого взаимодействия между приложениями, развернутыми в контейнерах

Изоляция Docker Container

Любой контейнер изолирован от других контейнеров и от хоста, на котором он запущен. Никакое действие или бездействие внутри контейнера не повлияет на другие контейнеры или Docker сервер. Можно удалить, например, пакет php7 из одного контейнера, но это никак не повлияет на другие контейнеры, в которых этот пакет стоит.

Примечание: это также не повлияет и на образ. Поэтому перезапустив контейнер, мы обнаружим, что пакет php7 снова окажется на месте.

Альтернативы Docker

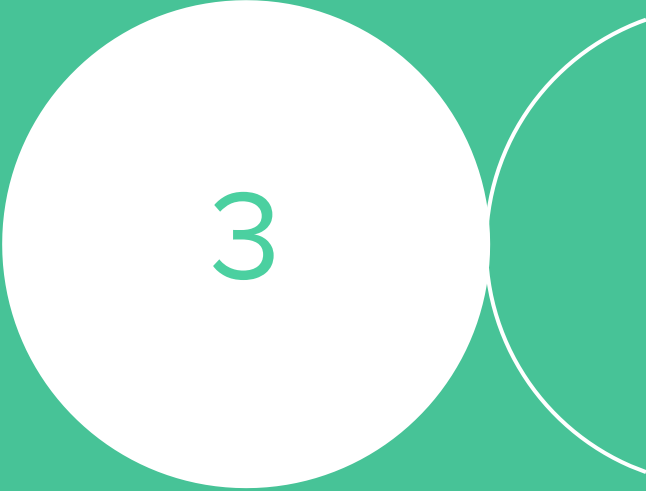
Аналоги со схожей целью (виртуализация):

- VirtualBox
- Parallels
- Vagrant

Аналоги со схожим функционалом (контейнеризация):

- podman
- containerd
- Kaniko
- runc/crun

Dockerfile



3

Dockerfile

Docker Hub хранит в себе множество готовых образов для самых различных целей. Но когда нужно сделать что-то своё, мы прибегаем к написанию Dockerfile.

Вместо того, чтобы каждый раз производить сборку вручную, мы один раз пишем сценарии. Указывая всё, что Docker необходимо сделать, чтобы на выходе получить единственно верный Docker Image

Dockerfile: пример работы с Apache2

Создадим этот Dockerfile и вставим в него содержимое со скрина:

```
# Используем, как основу, последний образ Debian
FROM debian:latest
# Указываем создателя образа
MAINTAINER Test Netology
# Указываем версию
LABEL version="1.0"
# Указываем команду, которая будет выполнена при сборке контейнера
RUN DEBIAN_FRONTEND="noninteractive" apt install -y tzdata && apt
update && apt install -y apache2 nano
# Обозначаем возможность прокидывать трафик на 80й TCP порт
EXPOSE 80/tcp
# Запускаем апач
CMD apachectl -D FOREGROUND
```

Примечание: Команда EXPOSE не прокидывает трафик с хоста в контейнер, а только говорит, что мы можем так делать

Dockerfile: пример работы с Apache2

Чтобы собрать из этого файла пригодный для работы образ, используем команду:

```
docker build -t netologytest1:1.1 .
```

После сборки найдём ID собранного контейнера с помощью команды:

```
docker image ls
```

Dockerfile: пример работы с Apache2

Запустим только что созданный образ с помощью команды:

```
docker run -d -p 80:80  
<id_созданного_образа/имя_созданного_образа>
```

Параметр -p говорит Docker Engine, что мы хотим с порта 80 Docker сервера прокинуть трафик на порт 80 контейнера.

Теперь можно в браузере открыть адрес `http://<ip_докерсервера>`. Мы увидим стандартную страницу, демонстрирующую, что Apache2 работает

Dockerfile: пример работы с Apache2

Кроме установки ПО внутри контейнера мы можем доставить внутрь контейнера свои файлы.

Заменяем стандартную страницу Apache своим файлом, добавив строку с командой COPY:

```
# Используем, как основу, последний образ
Debian FROM debian:latest
# Указываем создателя образа
MAINTAINER Test Netology
# Указываем версию
LABEL version="1.0"
# Указываем команду, которая будет выполнена при сборке контейнера
RUN DEBIAN_FRONTEND="noninteractive" apt install -y tzdata && apt update &&
apt install -y apache2 nano
# Копируем файл внутрь нашего контейнера
COPY ./index.html /usr/local/apache2/htdocs/index.html
# Включаем возможность прокидывать трафик на 80й TCP
порт EXPOSE 80/tcp
# Запускаем апач
CMD apachectl -D FOREGROUND
```

Dockerfile: пример работы с Apache2

Создадим рядом с файлом Dockerfile файл index.html:

```
nano ./index.html
```

Внутри файла укажем произвольный текст, например, Netology Docker Practice, сохраним и запустим сборку образа:

```
docker build -t netologytest1:1.2 .
```

Запустим только что созданный образ с помощью команды:

```
docker run -d -p 80:80 id_созданного_образа
```

Перейдя на адрес `http://<ip_докерсервера>`, мы увидим содержимое созданного нами файла index.html

Docker слои

Посмотрим на содержимое любого созданного нами образа. Для этого нужна команда:

```
docker image history id_образа
```

Мы увидим упорядоченные «слои», из которых состоит наш образ

Docker слои

- Мы можем взять любой готовый образ и использовать его, указав в строке FROM нашего Dockerfile. Тогда, не внося изменений в базовый образ, Docker поэтапно наложит все наши модификации на него при создании нового образа
- Слои позволяют не создавать каждый раз весь образ целиком, а пересоздавать только слой, относящийся к инструкции, которую мы изменили, и все идущие следом инструкции
- Слои можно рассматривать, как «коммиты» в репозитории. Они описывают процесс поэтапной модификации образа, из-за которой он пришёл к своему текущему состоянию

Docker слои

Подход, основанный на наложении слоёв, позволяет не тратить время на повторную сборку слоёв, которые не были затронуты очередным изменением.

Если попытаться повторно собрать образ из DockerFile, в который не было внесено изменений, сборка произойдёт почти моментально.

Примечание: Целесообразно часто меняющиеся элементы (к примеру, копирование файлов) помещать в конец Dockerfile, так сборка будет происходить быстрее

Начало работы с Docker



4

Docker Engine — поддерживаемые ОС

Docker Engine — основа системы. Docker предоставляет .deb и .rpm пакеты для систем, указанных в таблице ниже:

Платформа	x86_64/amd64	ARM	ARM64/AARCH64
CentOS	+		+
Debian	+	+	+
Fedora	+		+
Ubuntu	+	+	+

За самым актуальным способом установки docker в нужную систему лучше обращаться к [официальной документации](#)

Установка Docker Engine на Debian

Устанавливаем пакеты для работы apt через HTTPS

```
# Добавление GPG ключа для
репозитория sudo apt-get update
sudo apt-get install ca-certificates curl
gnupg sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg sudo chmod a+r /etc/apt/keyrings/docker.gpg

# Добавление репозитория для скачивания
пакетов echo \
  "deb [arch="$(dpkg --print-architecture)" signed-
by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/debian \
  "$(. /etc/os-release && echo "$VERSION_CODENAME")" stable" |
  \ sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Установка docker
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

Запускаем классический Hello World

Первое, с чего можно начать — это Hello World. Для этого используем команду для запуска образа hello-world:

```
docker run hello-world
```

Docker, не обнаружив образ hello-world локально, подключится к Docker Hub, скачает его и запустит контейнер. Об этом говорят характерные строки в выводе:

```
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
b8dfde127a29: Pull complete
Digest:
sha256:f2266cbfc127c960fd30e76b7c792dc23b588c0db76233517e1891a4e357d519
Status: Downloaded newer image for hello-world:latest
```

Запускаем классический Hello World

В результате мы увидим текст:

```
Hello from Docker!
```

И дополнительную информацию, которая разъясняет, что произошло, и даёт ссылки на документацию

Обновление контейнера

Например, если свой контейнер вы взяли из репозитория, hello-world, то при выходе обновления вам понадобится команда pull

```
docker pull hello-world
```

Смотрим список локальных Docker Images

По аналогии с командой `ls`, которая отображает содержимое каталога. Чтобы посмотреть хранимые локально Docker Images, можно использовать команду:

```
docker images
```

Результат будет представлен в виде текстовой таблицы:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
hello-world	latest	d1165f221234	2 months ago	13.3kB

Смотрим список Docker Container

Узнать список работающих контейнеров можно командой:

```
docker container ls
```

Смотрим список Docker Container

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
PORTS	NAMES			
ca424f568436	hello-world	"/hello"	4 seconds ago	Exited (0) 3 seconds ago
confident_bouman				
d289de3f4720	hello-world	"/hello"	6 seconds ago	Exited (0) 5 seconds ago
great_montalcini				
78768d88fb52	hello-world	"/hello"	30 minutes ago	Exited (0) 30 minutes ago
laughing_chandra				
d5cd52db2068	hello-world	"/hello"	31 minutes ago	Exited (0) 31 minutes ago
bold_taussig				

Заголовки в выводе списка Docker Container

- CONTAINER ID — идентификатор контейнера
- IMAGE — образ, использованный при создании контейнера
- COMMAND — запущенная команда
- CREATED — время, прошедшее с момента запуска
- STATUS — статус контейнера: работает или завершил работу
- PORTS — проброшенные в контейнер порты
- NAMES — случайно сгенерированное имя

Смотрим логи Docker Container

Узнав ID контейнера, можно посмотреть его логи с помощью команды:

```
docker logs <id_контейнера>
```

или

```
docker logs <имя_контейнера>
```

Docker run или Docker start

Узнав ID контейнера, можно посмотреть его логи с помощью команды:

```
docker run <имя_образа/id_образа>
```

Находит или скачивает образ, создаёт из него контейнер и запускает созданный контейнер. 10 раз запустив эту команду с именем какого-то образа, вы создадите 10 контейнеров:

```
docker start <id_контейнера/имя_контейнера>
```

Запускает уже существующий, но остановленный контейнер. Таким образом, вы можете запустить контейнер, произвести в нём настройки, остановить с помощью команды `docker stop` и потом запустить, не потеряв произведённые настройки

Удаляем лишние образы

Если необходимо удалить образ, чтобы освободить место, нужно сначала удалить все созданные на его основе контейнеры. Удалим, например, hello-world. Сначала смотрим имеющиеся контейнеры:

```
docker container ls -a
```

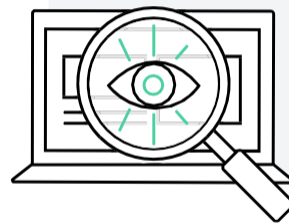
Смотрим на ID или имена и с их помощью удаляем контейнеры:

```
docker container rm <id_контейнера/имя_контейнера>
```

Когда все контейнеры, использующие образ, удалены, можно удалить сам образ

```
docker image rm hello-world
```

Демонстрация работы



Итоги занятия

Сегодня мы

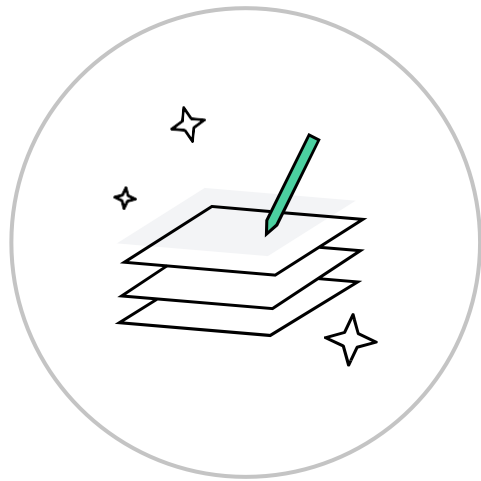
- Узнали, что такое Docker и для чего он используется
- Изучили основные компоненты экосистемы Docker
- Узнали как установить Docker и запустить тестовый контейнер hello-world
- Рассмотрели, как управлять образами и контейнерами Docker



Домашнее задание

Давайте посмотрим ваше домашнее задание.

- 1 Вопросы по домашней работе задавайте в разделе "Вопросы по заданию"
- 2 Задачи можно сдавать по частям
- 3 Зачёт по домашней работе ставят после того, как приняты все задачи



Дополнительные материалы

- [Официальная документация](#)

Режим easy:

- [Команда Selectel про докер](#)

Режим normal:

- [Список инструкций \(команд\) Dockerfile](#)
- [Best practices по написанию Dockerfile](#)

Режим hard:

- [Как повысить безопасность Docker-контейнеров](#)



Задавайте вопросы
и пишите отзыв о лекции

